# CSRF 취약점 진단 및 대응

01 CSRF 취약점 이해

02 CSRF 취약점 진단

03 CSRF 취약점 방어

# 📌 학습 목표

- ✅ CSRF(크로스 사이트 요청 위조)의 개념과 공격 원리를 이해한다.
- ✅ 실제 CSRF 공격 기법을 학습하고, 취약점을 진단하는 방법을 익힌다.
- ✅ CSRF 방어 기법을 적용하여 웹 애플리케이션 보안을 강화할 수 있다.

# 📍 학습 내용

1. CSRF 취약점 이해
2. CSRF 취약점 진단
3. CSRF 취약점 방어

# CSRF 취약점 진단 및 대응

01 CSRF 취약점 이해

02 CSRF 취약점 진단

03 CSRF 취약점 방어

# CSRF(크로스 사이트 요청 위조)란?

## 정의

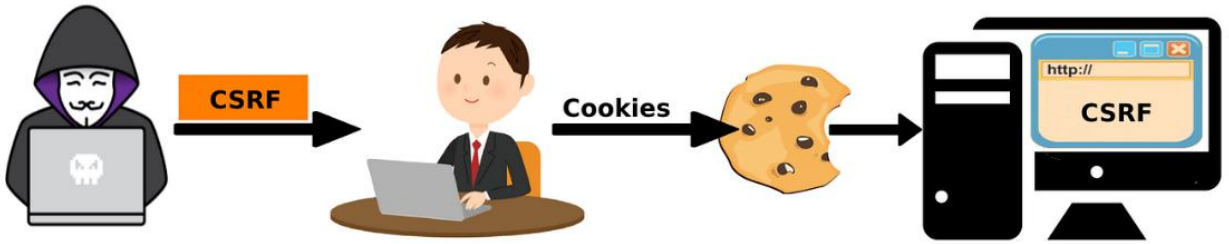
Cross-Site Request Forgery의 약자로, 사용자의 인증 정보를 이용해 의도치 않은 요청을 강제 실행하는 공격

→ 공격자가 악의적인 웹 페이지, 이메일, 또는 다른 매체를 통해 사용자의 브라우저에 요청을 보내도록 유도

## 취약점 배경

웹 애플리케이션이 인증 상태(쿠키 등)를 기반으로 요청을 처리하는 구조

브라우저는 같은 도메인에 대한 요청 시 자동으로 쿠키를 전송하므로, 사용자가 의도치 않은 요청에도 인증 정보를 포함하게 됨



# CSRF 취약점 발생 원인

인증 정보 자동 전송

브라우저는 사용자가 로그인한 사이트의 쿠키를 모든 요청에 자동으로 첨부함

→ 공격자가 악의적인 페이지에서 동일 도메인의 요청을 보내면, 사용자 인증 정보가 함께 전송되어 정상 요청으로 인식됨

요청 검증 부재

서버 측에서 요청의 출처(origin)나 `referer`를 검증하지 않거나, CSRF 토큰과 같은 추가 검증 절차 미적용

GET 요청 등 상태 변경에 사용되는 요청이 별도의 확인 없이 처리되는 경우

취약한 쿠키 설정

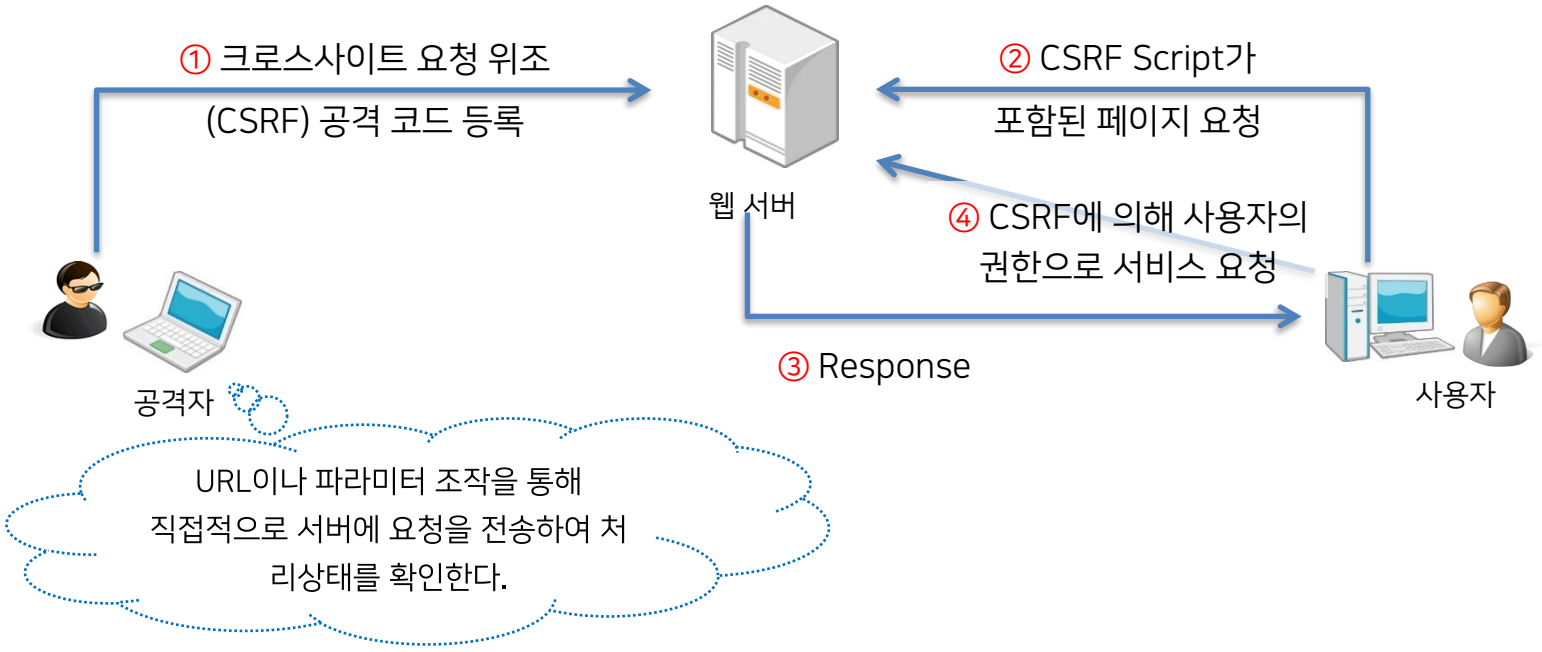
쿠키에 `SameSite` 속성이 적용되지 않으면, 타 도메인에서의 요청에도 쿠키가 전송될 수 있음

# CSRF 취약점 공격 흐름

사이트가 신뢰하는 사용자(인증된 사용자)의 세션을 통해, 공격자가 원하는 명령(공격자에게는 인가되지 않은)을 대신 실행하도록 한다.

페이지에 숨겨놓은 악의적인 공격(form문과 자바스크립트를 이용)으로 원하지 않는 악의적인 액션을 서버로 요청한다.

예를 들면, 숨겨둔 공격코드를 적은 게시판 글을 관리자가 누르면 관리자의 권한으로 form문을 실행시켜 어떤 액션을 수행하는 것이다.



# CSRF 취약점 공격 흐름

1. 사용자가 중요한 웹 서비스(예:은행, 증권)로 로그인

   세션 쿠키가 사용자 웹 브라우저에 저장됨

2. 악성 링크 클릭

   이메일 또는 게시판에 게시된 가짜 사이트 주소를 클릭

   · 이미지 로드시 동작

   ```
   <img src="https://bank.com/transfer?to=hacker&amount=1000000">
   ```

   · 숨겨진 iframe 로드시 무단 '좋아요' 클릭 동작

   ```
   <iframe src="http://sns.com/like?postId=434">
   ```

3. 공격자 사이트가 사용자의 웹 브라우저를 통해 로그인 되어 있던 중요한 웹 서비스에 요청 전송

   · 자동으로 저장되어 있던 세션 쿠키를 해당 사이트로 전송

4. 중요한 웹 사이트는 정상 사용자의 요청이라고 인식하므로 민감한 동작(예:계좌이체) 수행

# CSRF와 XSS의 차이점

XSS(크로스 사이트 스크립팅)

- 공격자가 웹 사이트에 악성 스크립트를 삽입, 사용자의 웹 브라우저에서 실행
- 주로 브라우저 내 스크립트 조작 및 쿠키 탈취, UI 변조에 활용

CSRF(크로스 사이트 요청 위조)

- 공격자가 사용자 대신 다른 웹 사이트에 요청을 보내도록 유도
- 서버 입장에서 사용자 의도와 무관한 요청을 정상 요청으로 오인



# CSRF 공격 사고 사례

시큐리티월드

보안뉴스

로그인 | 회원가입 | 기사제보 | 스크랩

통합검색

Home > 전체기사

## 워드프레스의 플러그인에서 위험한 CSRF 취약점 발견돼

입력 : 2020-04-29 17:12

**실시간 찾기 및 바꾸기 취약점...HTML 콘텐츠 쉽게 교체하게 해주는 플러그인**
**CSRF 취약점 토해 공격자들도 HTML 콘텐츠 바꿀 수 있어...4.0.2로 버전 업 필요**

[보안뉴스 문가용 기자] 워드프레스의 플러그인 중 '실시간 찾기 및 바꾸기(Real-Time Find and Replace)'라는 이름의 플러그인에서 고위험군 취약점 하나가 최근 발견됐다. 익스플로잇에 성공할 경우 웹사이트에 임의의 코드를 주입하는 게 가능하게 된다고 한다.

'실시간 찾기 및 바꾸기' 플러그인은 워드프레스 사이트 관리자들을 위해 설계된 것으로, 테마나 다른 플러그인 등과 같은 HTML 콘텐츠를 관리자들이 원하는 대로 교체할 수 있게 해준다. 오픈소스로 공개되어 있는 상태이며, 현재까지 약 10만 번 다운로드 되었을 정도로 워드프레서 생태계에서는 양호한 인기를 보여준다.

이 플러그인에서 발견된 취약점은 일종의 사이트 간 요청 위조(Cross-Site Request Forgery, CSRF) 취약점으로 분류되며, XSS 공격을 가능케 한다고 한다. 성공적으로 익스플로잇 할 경우, 공격자가 악성 자바스크립트 코드를 원하는 워드프레스 사이트에 주입할 수 있게 된다. 다만 해당 사이트의 관리자가 특정 링크를 클릭하거나 파일을 다운로드 받게 하는 등의 '속임수'가 성공해야 한다.

이 취약점을 발견한 워드프레스 보안 전문 회사인 데피안트(Defiant)는 "'실시간 찾기 및 바꾸기' 플러그인의 핵심 기능 중 하나인 '찾기 및 교체 규칙 추가하기'는 far_options_page라는 함수 내에 존재하는데, 이 함수는 요청의 출처를 제대로 확인하지 않는다는 약점을 가지고 있다"고 설명한다. 이것이 CSRF 취약점의 정체다.

# CSRF 취약점 진단 및 대응

01 CSRF 취약점 이해

02 CSRF 취약점 진단

03 CSRF 취약점 방어

# CSRF 공격 기법

CSRF 공격의 기본 원리

공격자가 악의적인 웹 페이지나 이메일 등을 통해 피해자의 브라우저로 타깃 사이트에 의도치 않은 요청을 보내도록 유도

→ 피해자의 인증 쿠키가 자동으로 포함되어, 서버는 정상 요청으로 처리

주요 공격 기법

· GET 요청 기반 공격

이미지 태그, 링크, 스크립트를 이용해 GET 요청을 강제 실행

(예) `<img src="https://target.com/deleteAccount?user=123" />`

· POST 요청 기반 공격

숨겨진 폼과 자바스크립트를 이용해 POST 요청을 자동 제출

(예) HTML 폼을 auto-submit 처리하여 중요한 작업(예: 자금 이체)을 실행

# CSRF 공격 기법

## 예시 1: GET 요청을 이용한 CSRF

공격자 페이지 코드

```
<!-- 공격자가 피해자에게 보낼 악의적 링크 예시 -->
<img src="https://target.com/action?param=value" alt="image" />
```

피해자가 해당 페이지를 로드하면, 브라우저가 자동으로 GET 요청을 보내고, 서버는 피해자의 쿠키와 함께 요청을 처리

## 예시 2: POST 요청을 이용한 CSRF (숨겨진 폼 & 자동 제출)

공격자 페이지 코드

```
<form id="csrfForm" action="https://target.com/transferFunds" method="POST">
  <input type="hidden" name="amount" value="1000">
  <input type="hidden" name="toAccount" value="attackerAccount">
</form>
<script>
  // 페이지 로드 시 폼 자동 제출
  document.getElementById("csrfForm").submit();
</script>
```

피해자가 페이지를 방문하면 폼이 자동으로 제출되어, 피해자의 인증 쿠키를 포함한 POST 요청이 실행

# CSRF 공격 기법

예시 3: 링크 클릭을 유도하는 사회공학적 CSRF

공격자가 피해자에게 URL을 포함한 메시지(예: 이메일, SNS)를 보내어 클릭을 유도

(예) `https://target.com/updateEmail?newEmail=attacker@example.com`

피해자가 링크를 클릭하면, 피해자의 인증 정보와 함께 요청이 실행되어 이메일 변경 등의 작업이 이루어짐

# CSRF 공격이 가능한 취약한 웹 사이트

상태 기반 인증 사용

사용자 로그인 후, 세션 쿠키를 통해 인증 상태 유지

(브라우저는 모든 요청에 대해 자동으로 쿠키를 전송)

부적절한 요청 검증

CSRF 토큰 미사용 또는 검증 로직 부재

Origin 또는 Referer 헤더 검증이 없는 경우

GET/POST 불분명한 구분

상태 변경 작업(예: 계정 삭제, 이메일 변경, 자금 이체 등)이 GET 요청으로도 수행됨

POST 요청의 경우에도 별도의 CSRF 보호 없이 처리

# CSRF 공격이 가능한 취약한 웹 사이트

입력 데이터 검증 미흡

사용자 입력(폼 데이터, URL 파라미터 등)을 검증 없이 처리

공격자가 임의의 값을 주입할 수 있음

자동화된 요청 검증 미흡

자동 제출 폼 또는 URL 링크를 통한 요청 전송

· GET 방식: `<img src="https://vulnerable.com/deleteAccount?userId=123" />`

· POST 방식: 숨겨진 폼이 별도의 CSRF 보호 없이 자동 제출됨

민감 작업에 대한 보호 미흡

중요 작업(계좌 이체, 비밀번호 변경 등)을 위한 추가 확인 절차 부재

사용자 의도와 관계없이 요청이 실행될 수 있음

# CSRF 취약점 진단 및 대응

01 CSRF 취약점 이해

02 CSRF 취약점 진단

03 CSRF 취약점 방어

# CSRF 토큰 적용 및 활용

## Spring Security의 CSRF 보호

기본적으로 CSRF 보호 기능이 활성화되어 있으며, HttpSession에 CSRF 토큰을 저장

웹 폼에 자동으로 CSRF 토큰이 포함되어, 폼 제출 시 서버에서 토큰을 검증

토큰 검증 실패 시 요청이 차단되어, CSRF 공격을 방지

## CSRF 보호 동작 방식

· 서버에서 토큰 생성: 사용자 세션 생성 시, CSRF 토큰이 함께 생성됨

· 클라이언트에서 토큰 포함: 폼, AJAX 요청 등 모든 상태 변경 요청에 CSRF 토큰이 숨겨진 필드나 HTTP 헤더로 포함됨

· 서버에서 토큰 검증: 요청에 포함된 토큰과 세션에 저장된 토큰을 비교하여 유효성을 확인

## 유의 사항

REST API와 같이 상태 비저장(stateless) 환경에서는 별도의 토큰 처리 전략 필요

# CSRF 토큰 적용 및 활용

Spring Security Java Configuration 예시

```java
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .csrf()// CSRF 보호 활성화 (기본값)
            .and()
            .authorizeRequests()
                .antMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            .and()
            .formLogin()
                .loginPage("/login")
                .permitAll();
    }
}
```

HTML FORM에  CSRF 토큰 적용 (Thymeleaf 예시)

Thymeleaf 환경에선 자동으로 히든 필드로 추가됨

```html
<form th:action="@{/update}" method="post">
    <input type="text" name="data" />
    <!-- Spring Security가 자동으로 CSRF 토큰 숨은 필드로 삽입 -->
    <button type="submit">제출</button>
</form>
```

AJAX 요청에서 CSRF 토큰 사용

1) HTML 생성시 meta 태그에서 토큰 이름과 값을 넣어 보냄

2) 스크립트에서 meta 태그의 토큰 이름/내용을 가져와 포함시켜 전송

```html
<!-- CSRF 토큰 메타 태그 -->
<meta name="_csrf"        content="${_csrf.token}" />
<meta name="_csrf_header" content="${_csrf.headerName}" />

<script>
  // 메타 태그에서 토큰·헤더명 추출
  const token  = document.querySelector('meta[name="_csrf"]').content;
  const header = document.querySelector('meta[name="_csrf_header"]').content;

  // AJAX 요청 (fetch)
  fetch('/api/data', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      [header]: token
    },
    body: JSON.stringify({ hello: 'world' })
  });
</script>
```

# SameSite Cookie 정책 설정

## SameSite Cookie란?

브라우저에서 웹 요청 시 어떤 조건 하에 쿠키가 함께 전송될지 제어하는 속성
CSRF 공격 등 외부 사이트에서의 악의적 요청으로부터 보호

## 주요 속성 값

- Strict
  쿠키가 오직 동일 사이트 내 요청에만 전송됨
- Lax
  최초 타 사이트 접속하는 경우만 전송하고 그 외 크로스사이트 요청들엔 차단
- None
  모든 요청에서 쿠키 전송
  반드시 Secure 플래그와 함께 사용해야 함

## CSRF 대응 효과

다른 도메인으로부터의 요청에 쿠키가 자동으로 포함되지 않아, CSRF 공격 위험 감소

```
HTTP/1.1 200 OK
Set-Cookie: JSESSIONID=ABC123DEF456; Path=/; HttpOnly; SameSite=Strict
```

# SameSite Cookie 정책 설정

Spring Boot 설정 예시

application.properties 파일에 지정하는 경우

```
server.servlet.session.cookie.same-site=strict
```

Java Config를 사용하는 경우

```java
@Bean
public CookieSerializer cookieSerializer() {
    DefaultCookieSerializer serializer = new DefaultCookieSerializer();
    serializer.setSameSite("Strict"); // 'Lax' 또는 'None' 사용 가능
    serializer.setUseSecureCookie(true); // 'None' 설정 시 반드시 true
    return serializer;
}
```

Node.js (Express) 설정 예시

```javascript
app.use(cookieParser());
app.use((req, res, next) => {
  res.cookie('sessionId', 'your-session-value', {
    sameSite: 'lax',  // 'strict' 또는 'none' (none 사용 시 secure:true 필수)
    secure: true      // HTTPS 환경에서만 쿠키 전송
  });
  next();
});
```

# Origin/Referer 검증 및 세션 관리 보안 강화

## Origin/Referer 검증의 개념

HTTP 요청 헤더의 Origin/Referer 값을 확인하여, 요청이 올바른 출처(도메인)에서 발생했는지 검증
공격자가 외부 사이트에서 악의적 요청을 보내는 것을 사전에 차단하는 역할

## 필요성

CSRF 공격 및 핫링크(Hotlinking)와 같은 외부 위협을 감소시킴
민감한 기능(예: 결제, 계정 변경 등)에 대한 요청이 예상된 출처에서 발생했음을 보장

## 구현 방법 예시

### 서버 측 로직

요청이 들어올 때, Origin/Referer 헤더 값을 확인하고, 사전에 정의된 화이트리스트(예: https://www.예상도메인.com)와 비교

일치하지 않는 경우 요청 거부 혹은 에러 처리

(예시)

```
// 간단 CSRF 검사: Origin 또는 Referer 확인
String origin = request.getHeader("Origin");
String referer = request.getHeader("Referer");
if ((origin != null && origin.equals("https://www.예상도메인.com")) ||
    (referer != null && referer.startsWith("https://www.예상도메인.com"))) {
    chain.doFilter(request, response);
} else {
    response.sendError(HttpServletResponse.SC_FORBIDDEN, "CSRF 차단");
}
```



# 요약 정리

- CSRF는 로그인된 세션 쿠키를 악용해 사용자가 의도치 않은 요청을 보내도록 유도하는 취약점이다.
- 피싱 링크나 악성 웹페이지로 사용자를 유도해, 브라우저 자동 전송 쿠키를 이용한 요청이 발생한다.
- 이미지·폼·링크 등을 통해 계정 변경, 자금 이체 등 민감 동작이 자동 실행될 수 있다.
- CSRF 토큰·SameSite 쿠키·Referer/Origin 검증을 병행 적용해 공격 표면을 최소화해야 한다.

# 참고 문헌


- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어

# 감사합니다.




# 퀴즈

1. **다음 중 CSRF 공격이 성공하기 위해 반드시 필요한 조건은 ?**

   1) 중요 웹사이트에 로그인되어 쿠키를 보유하고 있어야 한다.
   2) 서버가 Referer 검증을 엄격하게 적용해야 한다.
   3) CSRF 토큰을 매 요청마다 발급해야 한다.
   4) 브라우저가 SameSite=Strict로 쿠키를 설정해 둬야 한다.

2. **다음 중 CSRF 방어 기법에 해당하지 않는 것은 ?**

   1) CSRF 토큰을 사용해 요청 시 고유 값 검증
   2) SameSite 쿠키로 외부 도메인에서 쿠키 자동 전송 제한
   3) Referer/Origin 검사로 출처가 올바른지 확인
   4) 암호화된 비밀번호로 로그인 처리

3. **CSRF 공격을 완화하기 위해 쿠키에 설정하는 속성 중, 외부 도메인에서 요청이 발생했을 때 쿠키가 자동 전송되지 않도록 제한하기 위해 사용하는 속성은 ?**

   (  )




# 퀴즈 정답

1. **다음 중 CSRF 공격이 성공하기 위해 반드시 필요한 조건은 ?**

   1) 중요 웹사이트에 로그인되어 쿠키를 보유하고 있어야 한다.

   공격이 성립하려면, 사용자가 로그인 중이라 브라우저가 쿠키를 전송해야 합니다.

2. **다음 중 CSRF 방어 기법에 해당하지 않는 것은 ?**

   4) 암호화된 비밀번호로 로그인 처리

   비밀번호 암호화는 인증 과정 보안이지만, CSRF 방어 기법과는 직접 관련이 없습니다.

3. **CSRF 공격을 완화하기 위해 쿠키에 설정하는 속성 중, 외부 도메인에서 요청이 발생했을 때 쿠키가 자동 전송되지 않도록 제한하기 위해 사용하는 속성은 ?**

   (  SameSite  )